

Doctor & Patient Management System

Spring Boot + JWT + Role Based Authentication + MySQL

Technologies Used

- Java 17
 - Spring Boot 3
 - Spring Security 6
 - JWT Authentication
 - MySQL
 - Spring Data JPA
 - Maven
 - Postman
-

Project Features

- JWT Login Authentication
 - Role Based Access
 - Doctor CRUD APIs
 - Patient CRUD APIs
 - MySQL Integration
 - Spring Security Configuration
 - Validation Support
 - REST APIs
-

Package Structure

```
com.sanjit.dpms
|
├─ controller
├─ entity
├─ repository
├─ security
├─ config
```

```
|— dto
|— DoctorPatientJwtApplication.java
```

Step 1: Create Database

```
CREATE DATABASE doctor_patient_db;
```

Step 2: pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
```

```
<version>0.11.5</version>
</dependency>
```

Step 3: application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/doctor_patient_db
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

app.jwt.secret=abcdefghijklmnopqrstuvwxyz123456789
```

Doctor Entity

```
@Entity
@Table(name="doctors")
public class Doctor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String doctorName;
    private String specialization;
    private String email;
}
```

Patient Entity

```
@Entity
@Table(name="patients")
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;

private String patientName;
private int age;
private String disease;
}
```

JWT Utility

```
@Component
public class JwtUtil {

    @Value("${app.jwt.secret}")
    private String secret;

    private Key getKey(){
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String username,String role){

        return Jwts.builder()
            .setSubject(username)
            .claim("role", role)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis()+86400000))
            .signWith(getKey(), SignatureAlgorithm.HS256)
            .compact();
    }
}
```

Security Configuration

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
```

```
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().permitAll()
            );

        return http.build();
    }
}
```

Login API

Endpoint

POST http://localhost:8080/api/auth/login

Request Body

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response

eyJhbGciOiJIUzI1NiJ9...

Doctor APIs

Get Doctors

GET http://localhost:8080/api/doctors

Add Doctor

```
POST http://localhost:8080/api/doctors
```

```
{
  "doctorName": "Dr Rahul",
  "specialization": "Cardiologist",
  "email": "rahul@gmail.com"
}
```

Patient APIs

Get Patients

```
GET http://localhost:8080/api/patients
```

Add Patient

```
POST http://localhost:8080/api/patients
```

```
{
  "patientName": "Amit",
  "age": 35,
  "disease": "Fever"
}
```

Postman JWT Steps

Step 1

Open Postman.

Step 2

Call Login API.

```
POST http://localhost:8080/api/auth/login
```

Step 3

Copy JWT Token.

Step 4

Open Authorization Tab.

Step 5

Select Bearer Token.

Step 6

Paste JWT Token.

Step 7

Access Protected APIs.

Role Based Access

Role	Access
ADMIN	Manage Doctors & Patients
DOCTOR	View Patients

Interview Questions

What is JWT?

JWT is JSON Web Token used for secure authentication.

What is Spring Security?

Spring Security secures Spring Boot applications.

Difference between Authentication and Authorization?

Authentication verifies identity. Authorization checks permissions.

What is Role Based Authentication?

Different users get different permissions based on roles.

How to Run Project

1. Create Database
 2. Import Project into STS/Eclipse
 3. Update Maven Project
 4. Run Main Class
 5. Test APIs using Postman
-

Conclusion

This project demonstrates enterprise-level backend development using Spring Boot, JWT Authentication, Role Based Security, MySQL, and REST APIs.